



**CENTRO UNIVERSITÁRIO DE MARINGÁ - UNICESUMAR**  
**ENGENHARIA DE SOFTWARE**  
ATIVIDADE DE ESTUDO PROGRAMADA (AEP) - 4º SEMESTRE

## **PROJETO MICROGESTOR: SISTEMA DE GESTÃO FINANCEIRA PARA MICROEMPREENDEDORES**

Documento de Arquitetura e Requisitos - 1º Bimestre

### **Equipe de Desenvolvimento:**

- Eduardo André A. de Sant'Ana - RA: 25016225-2
- Kauan Maniezo de Souza - RA: 25001222-2
- Patrick Takeo dos Santos Oda - RA: 25362798-2

ODS Vinculada: ODS 8 (Trabalho Decente e Crescimento Econômico)

Repositório GitHub: <https://github.com/eduuusantana/aep-microgestor>

### **1. DESCOBERTA E CONCEPÇÃO (PROBLEMA E ODS)**

Trabalhadores autônomos e microempreendedores que atuam no segmento de alimentação informal, como produtores e vendedores de marmitas e doces caseiros, enfrentam barreiras constantes no controle de suas finanças. Durante a etapa de levantamento e entrevistas, identificou-se que o registro das vendas ainda é realizado predominantemente em blocos de anotações ou cadernos de papel, o que resulta em uma perda frequente de informações, desorganização dos pedidos e mistura entre finanças pessoais e recursos do negócio. Além disso, a prática comum de conceber crédito informal (conhecido como 'venda fiada') gera esquecimento de valores a receber e inadimplência no reconhecimento das finanças, impedindo que o empreendedor tenha clareza se sua atividade está operando com lucro ou prejuízo.

Dessa forma, esse projeto alinha-se diretamente ao Objetivo de Desenvolvimento Sustentável 8 (ODS 8 - Trabalho Decente e Crescimento Econômico) da Organização das Nações Unidas (ONU). O propósito do software MicroGestor é disponibilizar uma aplicação simples, confiável e de uso ágil, capaz de organizar as entradas, despesas e saldo real do negócio sem a complexidade de planilhas eletrônicas convencionais ou sistemas muito

robustos, contribuindo para a sustentabilidade financeira e o desenvolvimento desses pequenos trabalhadores.

## 2. ESCOPO E REQUISITOS DO SISTEMA

Para solucionar os problemas diagnosticados, foram definidos os seguintes requisitos funcionais para a aplicação:

- **RF01:** O sistema deve permitir o cadastro, a consulta, a alteração e a exclusão de produtos do catálogo, registrando nome e preço base unitário.
- **RF02:** O sistema deve permitir o registro de operações de venda (entradas), incluindo um ou mais produtos cadastrados com suas respectivas quantidades.
- **RF03:** O sistema deve permitir o registro de despesas operacionais (saídas), tais como a compra de insumos e matérias-primas de produção.
- **RF04:** O sistema deve permitir registrar a forma de pagamento associada às transações financeiras, podendo ser Dinheiro, Pix, Cartão de Crédito ou Débito Pendente/Fiado, de acordo com o tipo de operação realizada.
- **RF05:** O sistema deve gerar um relatório de fechamento de caixa, calculando o total de entradas, o total de despesas e o lucro líquido final do período.

## 3. CRONOGRAMA DE EXECUÇÃO

Essa é a distribuição das atividades de desenvolvimento do 2º Bimestre entre os integrantes da equipe:

| Período / Sprint            | Módulo / Épico       | Atividade Planejada                                               | Responsável |
|-----------------------------|----------------------|-------------------------------------------------------------------|-------------|
| Sprint 1<br>(01/10 a 15/10) | Catálogo de Produtos | Modelagem da classe Produto e implementação das operações de CRUD | Patrick     |

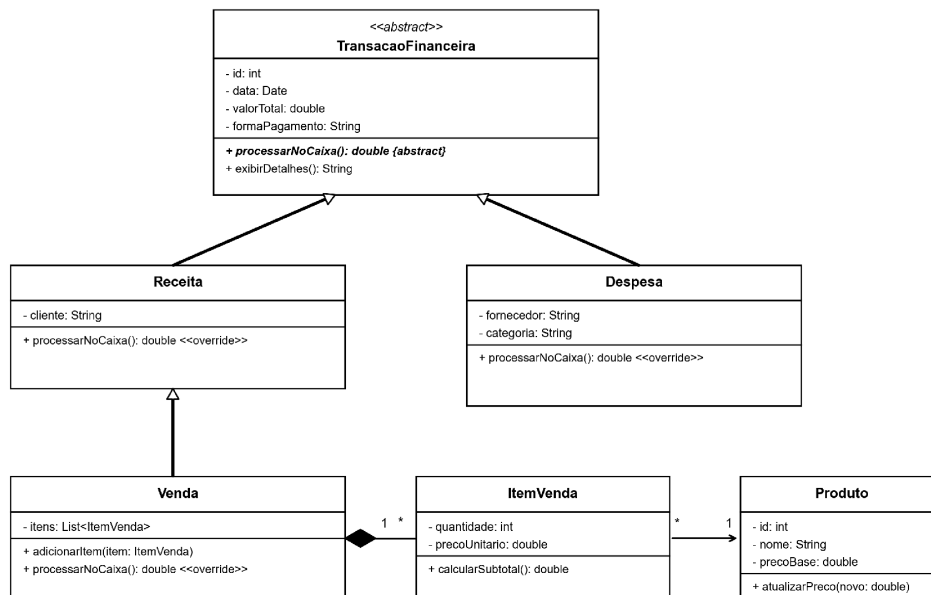
|                             |                     |                                                                                                         |         |
|-----------------------------|---------------------|---------------------------------------------------------------------------------------------------------|---------|
|                             |                     | (Create, Read, Update, Delete).                                                                         |         |
| Sprint 2<br>(16/10 a 30/10) | Motor Financeiro    | Construção da superclasse abstrata TransacaoFinanceira, classes filhas e métodos polimórficos de caixa. | Kauan   |
| Sprint 3<br>(01/11 a 15/11) | Banco de Dados      | Criação do script SQL das tabelas e implementação da conexão JDBC ativa com o banco de dados.           | Eduardo |
| Sprint 4<br>(16/11 a 25/11) | Relatórios e Testes | Implementação do fechamento de caixa, testes de integração entre camadas e documentação final.          | Todos   |

#### 4. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

A escolha das tecnologias foi alinhada conforme o conteúdo programático do semestre e na necessidade de construir um software estável e de fácil manutenção e implementação. Para o desenvolvimento no 2º Bimestre, será utilizada a linguagem Java. O Java foi selecionado por sua forte tipagem e conformidade com o paradigma da Programação Orientada a Objetos. Por lidar com cálculos financeiros e balanços contábeis, a segurança de tipos e o encapsulamento de dados são essenciais para evitar inconsistências nos valores apurados pelo sistema.

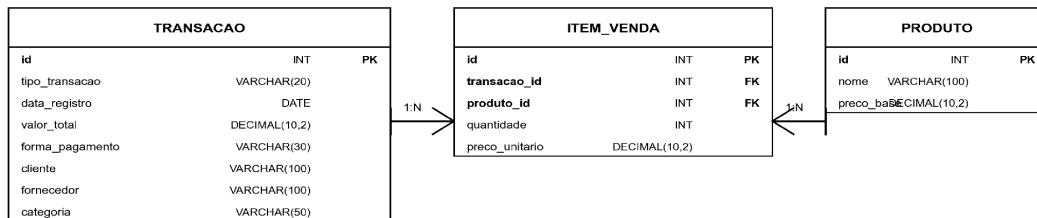
Para a camada de persistência, será utilizado um Sistema Gerenciador de Banco de Dados (SGBD) relacional utilizando a linguagem SQL para a criação, consulta e edição das tabelas. O modelo relacional foi escolhido pois assegura a integridade das transações por meio de restrições de integridade referencial (Foreign Keys), impedindo que dados inconsistentes sejam salvos e viabilizando o relacionamento 1:N entre a transação de venda e os respectivos itens vendidos.

## 5. DIAGRAMA DE CLASSES (MODELAGEM ORIENTADA A OBJETOS)



- **Herança e Abstração:** A classe abstrata `TransacaoFinanceira` reúne os atributos comuns a qualquer movimentação (`id`, `data`, `valorTotal` e `formaPagamento`) e serve de classe pai para as classes filhas `Receita` e `Despesa`. `Receita` adiciona o atributo `cliente`, enquanto `Despesa` adiciona `fornecedor` e `categoria`. A classe `Venda` herda de `Receita`, deixando o fluxo de atendimento aos clientes especializado.
- **Composição 1:N:** A classe `Venda` é composta por uma lista de `ItemVenda` (1 `Venda` contém 1 ou mais itens). Caso uma venda seja cancelada ou removida, seus itens associados deixam de existir conjuntamente.
- **Polimorfismo (@Override):** O método abstrato `'processarNoCaixa(): double {abstract}'` declarado na superclasse é sobrescrito polimorficamente (`@Override`) em cada subclasse: em `Receita` ele retorna um crédito positivo para o saldo (`+valor`), enquanto em `Despesa` ele debita o valor correspondente do caixa (`-valor`).

## 6. MODELAGEM DO BANCO DE DADOS (DER)



O modelo compõe-se de três tabelas principais, utilizando a estratégia Single Table para compatibilidade com o Diagrama de Classes: (1) PRODUTO: Armazena o catálogo de itens comercializados; (2) TRANSACAO: Unifica os registros de receitas e despesas por meio do atributo discriminador tipo\_transacao, armazenando os campos comuns (data\_registro, valor\_total, forma\_pagamento) e específicos (cliente para receitas; fornecedor e categoria para despesas); e (3) ITEM\_VENDA: Tabela que resolve o relacionamento entre a transação de venda e os produtos selecionados, guardando a quantidade e o preço praticado na operação.

## 7. REPOSITÓRIO GITHUB

- **/src:** Diretório reservado para o código-fonte da aplicação Java a ser desenvolvido no 2º Bimestre.
- **/docs:** Diretório destinado à documentação de engenharia de software e ao presente arquivo da AEP.
- **/database:** Diretório contendo o script SQL de criação das tabelas e dados iniciais de teste.

**Link público do repositório:** <https://github.com/eduuusantana/aep-microgestor>